

中山大学

《软件工程》期末大作业

软件工程化说明文档

项目名称：逸仙活动云——中山大学校园活动云平台

小组成员：钟梓俊 邱剑波 张宸睿

指导老师：王可泽

2026 年 7 月 16 日

目录

1 软件工程化目标 1

2 项目协作方式 1

2.1 团队分工 1

2.2 任务管理 1

2.3 沟通机制 2

3 版本控制规范 2

3.1 Git 仓库管理 2

3.2 分支策略 3

3.3 提交规范 3

3.4 合并请求与代码审查 3

4 开发规范 4

4.1 代码规范 4

4.2 目录规范 4

4.3 命名规范 5

4.4 文档规范 6

5 自动化手段 6

5.1 自动化构建 6

5.2 自动化检查 6

5.3 自动化测试 7

5.4 自动化打包 7

6 项目管理工具 7

7 工程化实践效果 8

7.1 已落地的工程资产 8

7.2 工程化实践的具体证据 9

7.3 持续改进方向 9

1 软件工程化目标

本项目以” 每次变更可追踪、可构建、可验证、可交接” 为工程化目标，围绕软件过程、协作、自动化和知识沉淀建立轻量但可执行的机制。核心不是堆砌工具，而是让需求—设计—代码—测试—发布形成闭环，并使前后端能在接口演化中保持一致。具体而言，工程化实践旨在达成以下目标：

- 1. **可追踪：** 每个功能从需求提出到上线发布的过程可追溯，变更历史可审计。
- 2. **可构建：** 每次提交均可通过自动化构建验证，类型错误和编译问题在合并前被发现。
- 3. **可验证：** 构建和测试过程自动化执行，减少人工回归成本，及早发现集成问题。
- 4. **可交接：** 环境配置和部署流程可复现，知识以文档形式沉淀，不依赖口头传递。

2 项目协作方式

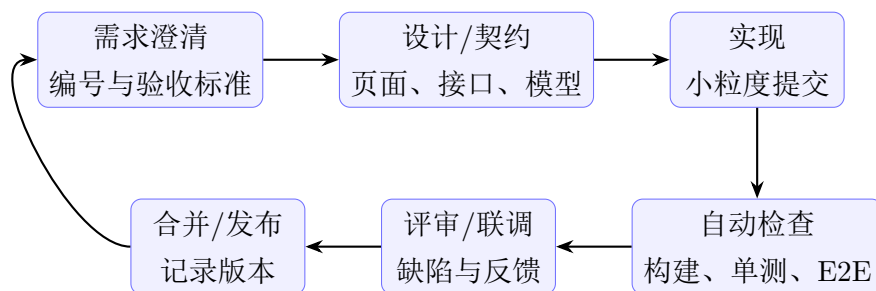
2.1 团队分工

本项目采用前后端分离的工作模式，团队成员分为前端、后端和测试：

组别	职责范围	工作产出
前端组	Vue 3 单页应用开发：页面组件（views）、通用组件（components）、路由配置（router）、状态管理（stores）、API 封装与数据适配（api）、可复用逻辑（composables）、工具函数（utils）。	前端源代码、页面设计文档、接口联调验证结果。
后端组	Flask REST API 开发：业务逻辑服务层（services）、数据模型（models）、认证授权（auth）、外部服务集成、部署配置。	后端源代码、数据库模型设计、API 接口实现。
测试组	接口契约制定与维护、端到端测试编写。	契约矩阵（BACKEND_CONTRACT_MATRIX.md）、Playwright E2E 测试。

2.2 任务管理

任务管理遵循” 需求 → 设计 → 实现 → 验证 → 归档” 的闭环流程：



每个任务由 GitHub Issue 承载，包含以下字段：需求编号（如 FR-03）、影响范围（涉及的前端页面、后端 API、数据库表）、完成定义（DoD）和风险说明。任务分配基于成员的技能领域和当前负载，优先保证 Must 功能的交付节奏。

2.3 沟通机制

团队采用以下沟通机制确保信息同步：

1. **接口联调**：前后端在开发新功能前共同确认接口字段、枚举值、分页参数和错误语义，更新契约矩阵。接口变更必须经双方确认并更新文档，仅在聊天中确认而未更新矩阵不得视为完成。
2. **代码审查**：每个合并请求至少由一名非作者成员审查，检查内容包括权限边界、异常路径、代码可读性和测试覆盖。审查未通过或 CI 失败的合并请求不得合并。
3. **变更通知**：数据库 Schema 变更、接口契约调整或环境配置修改在工作群中同步，确保前后端开发人员及时感知依赖变更。

3 版本控制规范

3.1 Git 仓库管理

项目分为两个独立仓库，职责清晰分离：

1. **前端仓库 (SYSU-Activity-Cloud-Frontend)**：包含 Vue 3 前端源码、端到端测试 (e2e/)、Mock 服务 (mock/)、CI 工作流 (.github/workflows/ci.yml) 和设计文档 (docs/)。
2. **后端仓库 (Campus-Activity-Information-Platform)**：包含 Flask 后端源码 (app/)、pytest 测试 (tests/)、Docker Compose 部署配置 (docker-compose.yml) 和 CI 工作流 (.github/workflows/)。

两个仓库在发布时以兼容版本对组成系统基线，记录前端 commit/tag、后端 commit/tag、接口契约版本和数据库 Schema 版本。主分支始终保持可构建状态，禁止直接推送。

3.2 分支策略

实际仓库已按以下分支策略运作：

1. **主分支 (main)**：保持可构建、可部署，所有变更通过合并请求引入。
2. **功能分支(feature/*)**：从 main 创建,每个分支只解决一个主题。示例:feature/auth-register、feature/activity-review。
3. **修复分支 (fix/*)**：用于缺陷修复，修复后合并回 main。

3.3 提交规范

提交信息遵循清晰描述变更目的的格式。实际仓库中的提交示例：

feat(auth)：添加邮箱验证码注册接口

- 实现验证码生成与发送接口 POST /api/auth/send-code
- 实现注册接口 POST /api/auth/register 含验证码校验
- 添加对应的 pytest 集成测试

fix(activity)：修复活动列表分页参数丢失问题

- 修复前端分页组件未传递 page 参数的 bug
- 添加分页参数的 E2E 测试覆盖

提交应小而聚焦，避免将多个无关功能的改动混入同一个提交。禁止提交真实密钥、编译产物 (dist/) 或本地数据库文件 (*.db)。

3.4 合并请求与代码审查

每个合并请求需包含以下三项内容：

1. **变更说明**：关联的需求编号 (FR-xx) 或缺陷编号，以及变更的简要描述。
2. **验证证据**：CI 运行链接或本地 npm run verify / pytest 的通过截图。
3. **文档更新**：接口契约、设计文档或运维文档若有变更需同步更新并附上链接。

审查者的检查清单包括：

1. 权限边界是否在服务端通过 JWT + RBAC 校验，前端是否隐藏或禁用越权入口。
2. 异常路径（空数据、网络失败、参数非法、越权访问）是否被覆盖。
3. 代码可读性：命名是否清晰、逻辑是否过度嵌套、注释是否必要且准确。
4. 测试覆盖：新增或修改的代码是否有对应的正向和异常测试用例。

CI 未通过、含真实密钥或破坏主干构建的合并请求禁止合并。紧急缺陷可走快速通道，但必须在修复后补齐测试与复盘记录。

4 开发规范

4.1 代码规范

1. **前端**: 使用 TypeScript 严格模式 (tsconfig.json 中 `strict: true`), 禁止使用 `any` 类型; 组件采用组合式 API (Composition API) 和 `<script setup>` 语法; 样式使用 CSS 类名, 避免内联样式; 遵循 Vue 官方风格指南。
2. **后端**: 遵循 PEP 8 编码规范; 视图函数 (蓝图路由) 不包含业务逻辑, 业务规则沉淀在服务层 (`services/`); 数据库操作通过 SQLAlchemy ORM, 避免裸 SQL。
3. **通用**: 不允许硬编码敏感信息 (密钥、密码、Token); 不允许提交调试代码 (`console.log`、`print`、断点); 不允许将编译产物和依赖缓存 (`node_modules/`、`.venv/`) 提交到仓库。

4.2 目录规范

前端目录结构按功能分层组织, 与实际仓库一致:

```
src/
  views/           页面级组件 (activity/ admin/ auth/ calendar/ 等)
  components/      可复用组件 (AppShell, NavHeader, PageState 等)
  api/             Axios 接口封装与数据适配
  stores/          Pinia 状态管理 (auth store 等)
  composables/     可复用组合式逻辑
  utils/           纯工具函数 (activitySort, activityQuality 等)
  router/          路由配置与导航守卫
  assets/          静态资源
  style.css        全局样式
```

后端目录结构按架构层次组织, 与实际仓库一致:

```
backend/
  app/
    __init__.py    应用工厂与扩展初始化
  blueprints/      Flask 蓝图模块
    auth.py        认证相关接口
    posters.py     活动 CRUD 与状态管理
```

search.py	搜索接口
calendar.py	个人日历接口
knowledge.py	知识节点与订阅
data_sources.py	数据源管理
review.py	审核管理
dicts.py	字典数据
home.py	首页推荐
export.py	数据导出
services/	业务服务层
quality_service.py	内容质量评分
dedup_service.py	疑似重复检测
audit_service.py	审计日志
notification_service.py	通知服务
security_service.py	URL 安全校验
models/	SQLAlchemy 数据模型
extensions.py	扩展初始化 (JWT, CORS 等)
config.py	配置加载
tests/	pytest 测试文件
conftest.py	测试夹具与配置
test_auth_api.py	
test_posters_api.py	
test_audit_logs_api.py	
... (20+ 测试文件)	

4.3 命名规范

1. **前端**: 组件文件使用 PascalCase (ActivityDetail.vue、LoginView.vue); 普通 TypeScript 文件使用 camelCase (activitySort.ts、activityQuality.ts); 路由路径使用 kebab-case (/activity/:id、/admin/review)。
2. **后端**: 蓝图模块使用 snake_case 文件名 (auth.py、data_sources.py); API 端点使用小写复数名词 (/api/activities、/api/auth/login); 模型类使用 PascalCase 单数形式 (User、Poster、Registration)。
3. **数据库**: 表名使用 snake_case 复数形式 (users、posters、registrations); 字段名使用 snake_case (created_at、password_hash)。

4.4 文档规范

项目文档统一使用 Markdown 格式，存放于 docs/ 目录。实际仓库中已存在的文档资产包括：

1. **页面设计文档 (docs/frontend-design/)**：共 8 份，每份包含页面结构、组件树、接口依赖和验收标准。覆盖登录、注册、首页、活动详情、活动编辑、搜索、日历、个人中心和后台管理页面。
2. **接口契约矩阵 (docs/frontend-plan/BACKEND_CONTRACT_MATRIX.md)**：表格记录前端字段名、后端字段名、类型、是否必填和示例值。
3. **联调检查清单 (docs/frontend-plan/BACKEND_INTEGRATION_CHECKLIST.md)**：按模块列出待联调接口、负责人和状态。
4. **开发规范 (docs/前端开发规范与流程.md)**：定义前端编码规范、提交规范和审查流程。
5. **功能需求文档 (docs/前端功能需求文档.md)**：详细描述各功能模块的页面行为和交互逻辑。

需求变更时同步更新相关的设计文档、接口契约和测试用例，避免文档与代码脱节。

5 自动化手段

5.1 自动化构建

前端通过 `npm run build` 触发 `vue-tsc -b && vite build`，串行执行 TypeScript 类型检查和 Vite 生产构建，输出可部署的静态资源到 `dist/` 目录。构建错误（类型不匹配、缺失引用）直接终止并输出错误位置，不生成无效构建产物。后端通过 Dockerfile 定义构建过程，安装 Python 依赖并打包 Flask 应用。

5.2 自动化检查

1. **类型检查**：`vue-tsc` 在每次构建时对 TypeScript 代码执行严格模式类型检查，在编译阶段捕获接口字段不匹配、空值引用和类型错误。
2. **代码质量**：后端 CI 计划集成 `pylint` 或 `flake8` 进行静态检查，当前阶段以前端构建时的类型检查和代码审查作为主要质量门禁。
3. **安全门禁**：代码审查中对敏感信息泄露、硬编码凭据、缺少输入校验和 SSRF 风险进行人工检查。

5.3 自动化测试

三项自动化测试已在项目中落地，集成于 `npm run verify` 命令：

1. **单元测试**：前端使用 Vitest + @vue/test-utils，覆盖认证 Store(`src/stores/auth.spec.ts`)、活动排序工具(`src/utls/activitySort.spec.ts`)、活动质量工具(`src/utls/activityQuality`)、日历卡片组件(`src/components/HomeCalendarCard.spec.ts`)和活动详情组件(`src/views/activi`)。后端使用 pytest + 内存 SQLite，覆盖认证、活动、审计、数据源、知识、搜索、质量、去重和安全等 20+ 测试文件。
2. **端到端测试**：使用 Playwright 驱动 Chromium 浏览器(`e2e/public-and-auth.spec.ts`)，覆盖访客浏览、登录注册、活动搜索等关键用户旅程。
3. **聚合验证**：`npm run verify` 串行执行构建、单元测试和端到端测试，作为提交前的完整性检查命令。

5.4 自动化打包

前端构建生成静态文件(`dist/` 目录)，可直接部署到 Nginx 或任何 Web 服务器。后端通过 Docker 构建不可变镜像，由 Docker Compose 编排启动。CI 工作流(`.github/workflows/ci.yml`) 在 push 和 PR 时自动触发，在 Ubuntu 环境中执行：

```
- uses: actions/checkout@v4
- uses: actions/setup-node@v4
  with:
    node-version: 22
    cache: npm
- run: npm ci
- run: npx playwright install --with-deps chromium
- run: npm run build
- run: npm run test:unit
- run: npm run test:e2e
```

6 项目管理工具

项目使用以下工具支撑工程化实践，每项工具均有对应的仓库证据：

1. **GitHub**：代码托管和版本控制。前端和后端分属两个独立仓库，主分支受保护，所有变更通过合并请求引入。
2. **GitHub Actions**：持续集成平台。前端仓库已配置 CI 工作流(`.github/workflows/ci.yml`)，在 push 和 pull request 上自动执行安装、构建、单元测试和端到端测试。工作流定义包含 Node 22 环境配置、npm 缓存和 Chromium 安装步骤。

- 3. **Docker Compose:** 后端部署编排工具。`docker-compose.yml` 定义 API 服务、PostgreSQL、Redis、Celery Worker 和 Celery Beat 的容器化拓扑,单命令(`docker compose up -d --build`) 启动完整后端环境。
- 4. **Mock 服务:** 基于 Node.js + Express 的轻量级 API Mock (`mock/index.js`), 实现认证、活动等核心接口的模拟响应, 供前端在无后端环境时独立开发和联调。
- 5. **Package Scripts:** 前端 `package.json` 定义了完整的脚本体系——`dev` (开发服务器)、`build` (类型检查 + 生产构建)、`test:unit` (Vitest)、`test:e2e` (Playwright)、`verify` (聚合检查)。

7 工程化实践效果

7.1 已落地的工程资产

工程实践	仓库证据	当前状态
前端构建与类型检查	<code>package.json</code> 中 <code>build</code> 脚本 (<code>vue-tsc -b && vite build</code>)	已配置并集成到 CI
前端单元测试	<code>src/</code> 下 5 个 <code>.spec.ts</code> 文件	已配置并集成到 CI
前端 E2E 测试	<code>e2e/public-and-auth.spec.ts</code> (Playwright)	已配置并集成到 CI
前端 CI 工作流	<code>.github/workflows/ci.yml</code> (push/PR 触发)	已实现自动化运行
前端设计文档	<code>docs/frontend-design/</code> 下 8 份设计文档	已形成文档资产
开发规范文档	<code>docs/前端开发规范与流程.md</code>	已建立
后端测试资产	<code>backend/tests/</code> 下 20+ 测试文件	测试代码存在
后端部署配置	<code>Dockerfile</code> 、 <code>Compose</code> 、 <code>Gunicorn</code> 、 <code>.env.example</code>	已配置作为部署起点

7.2 工程化实践的具体证据

1. **CI 运行记录：** GitHub Actions 工作流在 push 和 pull request 时自动触发，执行构建、单元测试和端到端测试。CI 配置缓存 npm 依赖加速安装，Playwright 自动下载 Chromium 浏览器。
2. **设计文档一致性：** 8 份前端设计文档覆盖全部主要页面，每份包含页面结构、组件树、接口依赖和验收标准，与最终实现的视图组件和 API 接口可对应验证。
3. **契约矩阵维护：** 接口契约矩阵已建立并随迭代更新，前后端字段映射、类型和示例值在联调前达成一致，减少联调阶段的字段不匹配问题。
4. **Mock 服务联调：** Mock 服务实现认证和活动等核心接口的模拟响应，前端可在不启动后端的情况下独立验证页面交互和状态管理逻辑。

7.3 持续改进方向

以下项目被明确列为下一步改进计划，不做为已完成能力表述：

1. 新建后端 CI 工作流：安装开发依赖后执行 pytest、静态检查（pylint/flake8）和覆盖率报告。
2. 将契约矩阵纳入自动化契约测试，阻止字段映射、枚举值和分页参数的回归。
3. 以受保护主分支、必需评审和必需 CI 状态实施分支保护策略，确保合并质量。
4. 发布流水线增加镜像版本标签、健康检查、数据库迁移自动执行和回滚步骤。
5. 用 Issue 看板统计缺陷率、返工率和交付周期数据，在迭代回顾中驱动过程改进。